K-TH ELEMENT OF TWO SORTED ARRAYS

ALGORITHM PROJECT 7

Prepared by :

- Nagham Reda Ibrahim – 20241058
- Sama Sherif Elsayed Mohamed – 20240434
- Mariam Ahmed Mohamed Ahmed – 20240929
- Jowaireya Ahmed Mohamed Tag – 20240264
- Rana Rashad Yahia Abdelatty –20240356
- Maya Ahmed Mohamed – 20240775
- Omar Mahmoud Gomaa – 20240653

Faculty of Computing & Artificial Intelligence, Capital University

(Algorithms- 2025/2026 – Level 2- Semester 2)

Comparison and analysis

First Algorithm – Non Recursive (Iterative)

This algorithm merges the two sorted arrays by comparing elements one by one until the k-th element is found. It behaves similarly to the merge step in Merge Sort.

1. Pseudo code

```
Function kthElementMerge(A, B, k):
    i ← 0   // pointer for array A
    j ← 0  // pointer for array B
    count ← 0

    While i < length(A) AND j < length(B):
        If A[i] < B[j]:
            count ← count + 1
            If count == k:
                return A[i]
            i ← i + 1
        else:
            count ← count + 1
            If count == k:
                return B[j]
            j ← j + 1

    while i < length(A):
        count ← count + 1
        if count == k:
            return A[i]
        i ← i + 1

    while j < length(B):
        count ← count + 1
        if count == k:
            return B[j]
        j ← j + 1

    return -1
```

2. Implementation (Java)

Provided in KthElement.java– uses two pointers and counts until k elements are taken.

3. Complexity Analysis (with step by step)

Time Complexity

- Each loop iteration takes exactly one element from either array.
- We stop when count == k .
- Number of iterations = k (in the worst case, k = m + n).
- Each iteration of the loop does O(1) work (one comparison, one increment, one equality check)
- Therefore **T(n) = O(k).**

- If k ≈ n+m (near the end of both arrays), the complexity approaches **O(n + m)**

Second Algorithm – Recursive (Binary Discard / Divide & Conquer)

1. Pseudo code

```
Function kthRecursive(A, B, k):
  Return helper(A, length(A), B, length(B), k)


Function helper(A, m, B, n, k):

  If m == 0:
      Return B[k - 1]           // A is empty, answer is in B
  If n == 0:
      Return A[k - 1]           // B is empty, answer is in A
  If k == 1:
      Return min(A[0], B[0])

  i = min(m, k / 2)             // Take at most k/2 elements from A
  j = min(n, k / 2)             // Take at most k/2 elements from B

  If A[i - 1] < B[j - 1]:

      newA = A[i .. m-1]        // Copy A starting from index i
      Return helper(newA, m - i, B, n, k - i)

  Else:
      newB = B[j .. n-1]        // Copy B starting from index j
      Return helper(A, m, newB, n - j, k - j)
```

2. Implementation(Java)

Provided in KthElement.java – uses recursion to discard approximately k/2 elements

Each call

3. Complexity Analysis (with step by step)

Time Complexity

- In each recursive call, the algorithm removes nearly half of the elements from one of the arrays
- After each call, the remaining k reduces to roughly k/2
- The recursion depth is therefore O(log k).
- Each call does a constant amount of work: comparisons, arithmetic, and pointer adjustments.
- Hence **T(n) = O(log k).**

- Since $k \leq m + n$, this is also **O(log(m +n)).**

Comparison Between the Two Algorithms

Criterion	Non recursive (Iterative)	Recursive Binary discard)
Time Complexity	O(k) → worst O(m+n)	O(log k) → worst O(log(m+n))
Best Case	k=1 → 1 step	k=1 → O(1)
Worst Case	k=m+n → m+n steps	k=m+n → $\log_2$ (m+n) steps
Implementation Complexity	Simple, linear merge	More complex, careful index handling
Handling Invalid k	Returns -1 if k > m+n or k <1	Checked in main function before calling algorithm
Recursion Limit	Not applicable	Uses recursion stack which may increase memory usage
When to Prefer	Small k, or when simplicity and constant memory are needed	Large k, large arrays, performance critical scenario

Which Algorithm Is Better and Why?

The recursive algorithm is generally better than the non-recursive merge algorithm because it is more efficient for large inputs.

The merge method checks elements one by one until reaching the k-th element, so its time complexity is: **O(k)**

the recursive algorithm removes nearly half of the remaining elements in each recursive call using the divide-and-conquer technique. Therefore, its time complexity is: **O(log k)**

This makes the recursive solution significantly faster, especially when the arrays are large.

the non-recursive algorithm is easier to understand and implement, making it more suitable for beginners and small datasets.